

HW1: A Bounded Agent for Bandit Alert Classification

COMS E6998 Section 019, Fall 2026 · Design of Production Agentic Systems

Instructor: Rahul Krishna · TA: In Keun Kim

Released Friday, September 25, 2026 · Due Friday, October 16, 2026, before 11:59 p.m.

1 Overview

You will build a vulnerability management agent. HW1 is limited to agentic scanning of a Python repository and classification of the alerts you find.

The HW handout fixes the interfaces you'll be designing to make them consistent and testable:

- **The command line:** how you invoke your agent from the terminal.
- **The configuration:** the settings your agent reads from the config file.
- **The output file:** where your agent writes the results of its analysis.
- **The trace:** the detailed record of the agent's execution.
- **The README:** the documentation that explains how to use your agent.

You design and build everything behind them. You get one point for each stage of the agent, so your grade shows which mechanisms exist and work.

Grades assess only the implementation of the required agent mechanisms, not classification correctness or performance. Classification results are collected for teaching purposes only. Submit the design report and implementation together; the report and README document and provide evidence of the implementation. There is no separate design checkpoint. You may consult the instructors as often as you like, at any stage of the work.

Released	Friday, September 25, 2026
Due	Friday, October 16, 2026, before 11:59 p.m.
Weight	20% of the course grade, marked out of 20 (Section 9)
Format	Solo/Team assignment. One submission per student/team.
Late days	4 per student per semester, at most 2 on one assignment; each extends the deadline by 24 hours.

2 What you build

Your agent takes one (pinned) Python repository and works in two stages.

1. **Discovery.** Run a tool called bandit (Python security linter, its a CLI tool, see <https://bandit.readthedocs.io/en/latest/>). Set it at **MEDIUM or higher severity** over the scan scope that the configuration sets for you (Section 4).
2. **Classification.** Label every alert **1** (potential vulnerability) or **0** (not a vulnerability), with a text rationale block that includes line evidence.

You then write a `report.json` file, with one row per alert, and record the agent's trace in a `trace.jsonl` file, the record of the run.

You write all of the code. The handout gives no starter code, stubs or tests. Any language and framework is allowed, but Bandit needs Python 3.12. The target code is read only: your agent reads it, and never installs or runs it.

We ship these files in this repository: `config.schema.json`, `report.schema.json` and `trace.schema.json` at the root; the three required `[scan]` blocks as `briefs/scan/radicale.toml`, `briefs/scan/sglang.toml` and `briefs/scan/owasp-benchmark-python.toml`; and the example output `briefs/report.example.json`.

3 Targets

Clone each target at its pinned commit. The root of that clone is `--input`. Your agent must run on all three targets.

Target	Repository and commit	Role	Alerts
OWASP BenchmarkPython	https://github.com/OWASP-Benchmark/BenchmarkPython f1291485808b66e20ddb6b01b10dc71b3df8c8ba	development	18 alerts
Radicale	https://github.com/Kozea/Radicale eff8027f3dc4910be1659ee71f4b4a454ade5a8c	held out	10 alerts
SGLang	https://github.com/sgl-project/sglang bbe9c7eeb520b0a67e92d133dfc137a3688dc7f2	held out	21 alerts

OWASP BenchmarkPython is the development set. Its answer key is open to you: the benchmark’s own labels, in the file `expectedresults-0.1.csv` at the repository root. Read it to measure your agent and to refine its prompts, tools and stopping rules. **Your agent may not read it.** At run time the key is withheld from the agent’s workspace (Section 7); an agent that reads it measures nothing.

Radicale and SGLang are held out. Their answer keys are hidden, and staff evaluate classification results after submission for teaching purposes only, with no effect on grades. They show whether your agent generalises or has only been tuned to the benchmark.

4 Required scan scope for each target

Required scan scope for each target

These three `[scan]` blocks are mandatory. Use them unchanged: graders run every submission with exactly these blocks. `paths` and `exclude` are globs relative to `--input`, and your Bandit wrapper expands them.

Radicale: 10 alerts

```
[scan]
paths = ["radicale"]
exclude = ["radicale/tests"]
min_severity = "MEDIUM"
bandit_version = "1.9.4"
```

SGLang: 21 alerts

```
[scan]
paths = ["python/sglang/multimodal_gen/runtime",
        "scripts/playground/replay_request_dump.py"]
exclude = []
min_severity = "MEDIUM"
bandit_version = "1.9.4"
```

OWASP BenchmarkPython: 18 alerts (test cases 00001 to 00099)

```
[scan]
paths = ["testcode/BenchmarkTest000*.py"]
exclude = []
min_severity = "MEDIUM"
bandit_version = "1.9.4"
```

The counts come from Bandit 1.9.4 on CPython 3.12, measured on fresh clones at the pinned commits. They count results of MEDIUM or higher severity, at any confidence.

From a `[scan]` block to Bandit flags

Your Bandit wrapper turns each key into a command-line flag. Run Bandit from the root of the target checkout (the `--input` directory).

[scan] key	Bandit command line
paths	Positional arguments, after glob expansion; <code>-r</code> recurses into directories.
exclude	<code>-x</code> with a comma-separated list; omit the flag when the list is empty.
min_severity = "MEDIUM"	<code>--severity-level medium</code> (the same as <code>-ll</code>).
(any confidence)	No <code>--confidence-level</code> flag; the default reports every confidence.
bandit_version	Install <code>bandit==1.9.4</code> ; <code>bandit --version</code> must report it.
(output)	<code>-f json -o <file></code> , which your wrapper parses. <code>-q</code> suppresses the banner.

The three required blocks translate to these commands. Each gives the alert count stated above. Bandit exits with status 1 when it reports results, so do not treat that exit status as a failure.

```
# Radicale (from targets/radicale)
bandit -q -r --severity-level medium -x radicale/tests -f json -o bandit.json radicale

# SGLang (from targets/sglang)
bandit -q -r --severity-level medium -f json -o bandit.json \
python/sglang/multimodal_gen/runtime scripts/playground/replay_request_dump.py

# OWASP BenchmarkPython (from targets/owasp-benchmark-python); the shell expands the glob
bandit -q -r --severity-level medium -f json -o bandit.json testcode/BenchmarkTest000*.py
```

Bandit adds a leading `./` to the paths of files named on the command line, as in the OWASP command. Strip that prefix before you write `report.json` (Section 6).

5 Interface

The agent is a command-line program:

```
./your-agent --input <target checkout root> --output <out dir> --config config.toml
```

It writes two files into `<out dir>`: `report.json` (Section 6) and `trace.jsonl` (Section 8). You choose the name `your-agent`, and your README states it.

Your agent must read the configuration file named by `--config`. It is a TOML file, and once parsed it must validate against `config.schema.json`. You ship two configs, a full budget and a low budget (Section 10), and we run your agent with them.

Key	Rule
<code>[scan]</code>	Exactly one of the three required blocks (Section 4), unchanged.
<code>model.provider, model.name</code>	Required strings: your provider and the exact model name.
<code>budget.max_tokens</code>	Required integer ≥ 0 : the token budget for the whole run, across all alerts.
<code>budget.output_allowance</code>	Required integer ≥ 1 : the maximum output tokens for each model call, sent to the provider.
<code>budget.max_steps</code>	Required integer ≥ 1 : a failsafe, separate from the token budget.
(other keys)	Allowed, for your own settings. Never put secrets in the file; read them from environment variables.

A valid config for Radicale:

```
[scan]
paths = ["radicale"]
exclude = ["radicale/tests"]
min_severity = "MEDIUM"
bandit_version = "1.9.4"

[model]
provider = "anthropic"
name = "claude-sonnet-5"

[budget]
max_tokens = 200000
output_allowance = 1024
max_steps = 200
```

The token budget covers the whole run, not each alert. The budget charges `input + output` as the provider reported them, summed over every `model_call` (Section 8). `cache_read` and `cache_write` are recorded when the provider reports them, and are never charged. `budget.used_tokens` in `report.json` is exactly that sum. When the budget runs out, every alert that is not yet classified still gets a row in `report.json`, with `status` set to `budget_exhausted`.

6 Output

`report.schema.json` (JSON Schema) is normative: your `report.json` must validate against it. The example below is a truncated illustration: it shows 3 of the 18 rows of an OWASP BenchmarkPython cutoff run, and its `alert_count` equals

its 3 rows. It shows the format only; the rationales are placeholders.

briefs/report.example.json (truncated illustration: 3 of the 18 OWASP rows)

```
{
  "target": {"input": "targets/owasp-benchmark-python", "commit": "f1291485808b66e20ddb6b01b10dc71b3df8c8ba"},
  "scan": {"bandit_version": "1.9.4", "min_severity": "MEDIUM", "alert_count": 3},
  "budget": {"max_tokens": 20000, "used_tokens": 19480, "exhausted": true},
  "alerts": [
    {
      "alert_id": "owasp-BenchmarkTest00073-py-54-B307",
      "test_id": "B307",
      "path": "testcode/BenchmarkTest00073.py",
      "line": 54,
      "severity": "MEDIUM",
      "confidence": "HIGH",
      "label": 0,
      "status": "classified",
      "rationale": "Format illustration only: one or two sentences citing what the agent read.",
      "evidence": [{"path": "testcode/BenchmarkTest00073.py", "start_line": 45, "end_line": 58}]
    },
    {
      "alert_id": "owasp-BenchmarkTest00074-py-51-B102",
      "test_id": "B102",
      "path": "testcode/BenchmarkTest00074.py",
      "line": 51,
      "severity": "MEDIUM",
      "confidence": "HIGH",
      "label": null,
      "status": "error",
      "rationale": "The model reply could not be parsed as a label.",
      "evidence": []
    },
    {
      "alert_id": "owasp-BenchmarkTest00078-py-53-B301",
      "test_id": "B301",
      "path": "testcode/BenchmarkTest00078.py",
      "line": 53,
      "severity": "MEDIUM",
      "confidence": "HIGH",
      "label": null,
      "status": "budget_exhausted",
      "rationale": "",
      "evidence": []
    }
  ]
}
```

Rules:

- There is exactly one row for each in-scope Bandit result. Rows are never dropped or merged, so `len(alerts)` equals `scan.alert_count`.
- `path` and every evidence `path` are repository-relative POSIX paths with no leading `./` (Bandit prints `./` for explicitly listed files; strip it). Rows are ordered by plain string comparison of `path`, then `line` as an integer, then `test_id`.
- `status` is one of `classified`, `budget_exhausted` or `error`.
- `label` is 1 or 0 when `status` is `classified`. Otherwise it is `null`.
- `budget.used_tokens` is exactly the budget charge (Section 5): `input + output` as the provider reported them, summed

over every `model_call`. `cache_read` and `cache_write` are never charged.

7 Workspace and withholding

You build the workspace and the withholding yourselves; there is no deny list in the configuration. The reference pattern is cyberbird's `cyberbird/reactive/workspace.py`, in particular `resolve`, `AgentWorkspace` and `WITHHELD_GLOBS`:

<https://github.com/comse6998-019/cyberbird/blob/cf7e96c8bd9e74470b99b7b754bb72392427cfc6/cyberbird/reactive/workspace.py>

Discovery (the Bandit run) scans the whole required `[scan]` scope. The withholding rules apply to what the model can read through the classification tools: `read`, `search`, and any other tool the model can call.

Requirements:

1. The agent works in a disposable copy of `--input`. The tools the model can call reach only that copy.
2. Every tool path goes through one path check. The check refuses `..` paths, absolute paths, and symlinks that resolve outside the copy.
3. `.git/` is withheld on every target. The history of a full clone can hold commits made after the pinned one, including fixes.
4. On OWASP BenchmarkPython, the answer key `expectedresults-0.1.csv` (in general, `expectedresults*.csv`) is withheld from the agent. So is every `BenchmarkTest*` file except the case under investigation, because each case has a safe twin. You may still read the key yourselves (Section 3); only the agent's view is restricted.

8 Trace and trajectory

The trace is the agent's trajectory: the complete, ordered record of what one run did and who decided each step. Graders read it to check B3 to B8, so its format is fixed. You build it yourselves. The reference implementation is cyberbird's `cyberbird/plan_and_validation/trace.py` (`Trace`, `EventKind`, `TerminalStatus`, `Usage`) and `cyberbird/plan_and_validation/trajectory.py` (trace to Mermaid sequence diagram):

https://github.com/comse6998-019/cyberbird/blob/cf7e96c8bd9e74470b99b7b754bb72392427cfc6/cyberbird/plan_and_validation/trace.py

https://github.com/comse6998-019/cyberbird/blob/cf7e96c8bd9e74470b99b7b754bb72392427cfc6/cyberbird/plan_and_validation/trajectory.py

Format:

- `<out dir>/trace.jsonl` holds one JSON object per line. It is append-only and flushed after every event, so that a crashed run leaves the trace that explains the crash.
- Every event has `step` (an integer from 1, increasing by 1), `ts` (Unix time, float), `run_id`, `alert_id` (the alert the event belongs to, or `null` for run-level events) and `kind`.
- `kind` is one of the closed set below. Any event may carry other fields as well.

kind	Written by	Required fields
<code>model_call</code>	the node that called the model	<code>role</code> , <code>model</code> , <code>usage</code> with <code>input</code> and <code>output</code> (integers, as the provider reported them; also <code>cache_read</code> and <code>cache_write</code> if the provider reports them; no summed total)
<code>tool_request</code>	the dispatcher	<code>tool</code> , <code>args</code>
<code>tool_result</code>	the dispatcher	<code>tool</code> , <code>ok</code> (boolean), and <code>error</code> when <code>ok</code> is false
<code>routing</code>	the edge function that chose the next node	<code>decision</code> , <code>by</code> (<code>model</code> or <code>runtime</code>)
<code>state_change</code>	any node, for updates not covered above	<code>node</code> ; when an alert's row is settled (<code>classified</code> , <code>error</code> , or <code>budget_exhausted</code> , including rows settled at a crash or at the cutoff), also <code>alert_status</code> and <code>label</code>
<code>terminal</code>	the runtime, once, last	<code>status</code> (one of your terminal statuses, A6), <code>usage_total</code> (an object with the same counters as <code>usage</code>)

Invariants (graders check these on every trace):

1. The file parses line by line, and every event has the required fields for its `kind`.
2. `step` starts at 1 and increases by exactly 1.
3. There is exactly one `terminal` event, and it is the last line, also for a run that crashed.

4. Each counter in `usage_total` equals the sum of that counter over all `model_call` events. `budget.used_tokens` in `report.json` equals exactly the sum of `input + output` over those events; `cache_read` and `cache_write` are recorded but never charged.
5. For every row in `report.json`, including `budget_exhausted` and crash-time `error` rows, there is exactly one `state_change` event with that row's `alert_id` whose `alert_status` and `label` equal the row's `status` and `label`.
6. Every `tool_request` is immediately followed by its `tool_result` (the dispatcher handles one request at a time), before the next `model_call`.

`trace.schema.json`, at the root of this repository, is normative for each line: every line of your `trace.jsonl` must validate against it. It encodes the kind table above; the invariants are rules across lines, which a schema cannot express, so graders check them separately. `briefs/trace.example.jsonl` is the trajectory behind `briefs/report.example.json`: it validates line by line and meets all six invariants.

9 Rubric

The assignment is marked out of 20, one point per item. These items assess the implementation of the required mechanisms and the documentation and evidence that demonstrate them, not the correctness or performance of classifications.

Part A: design report (7)

- A1 **Behaviours.** What the agent does for each alert, and what it must never do: run target code, write to the target, invent observations.
 - A2 **State machine.** A diagram of the states and the transitions between them.
 - A3 **Nodes and edges.** Each node, each edge marked as conditional or fixed, and who decides each transition (the model or the runtime).
 - A4 **Agent state.** Each field with its reducer, its lifetime, who writes it, and whether the model sees it.
 - A5 **Tools.** Contracts for at least 3 tools, with arguments, limits and refusal cases.
 - A6 **Start, stop and termination.** A closed, mutually exclusive set of terminal statuses and what triggers each one. This includes the budget policy: how the charge above is applied, when it is checked, how the output allowance is sent, and what happens at the cutoff.
 - A7 **Workspace and withholding.** Which files the agent must not see for each target and why, how they are withheld, and how they are restored.
-

Part B: implementation (10)

- B1 **Bandit tool.** Reads `[scan]`, uses MEDIUM+ severity, uses Bandit 1.9.4, and parses the JSON output.
 - B2 **Workspace, read and search tools.** Meets every requirement in Section 7.
 - B3 **Dispatcher.** The runtime validates and runs every request. Unknown tools and bad arguments come back as refusal observations, not crashes. Only the runtime writes observations.
 - B4 **Graph.** Implemented as the design in Part A describes.
 - B5 **Tracer and trajectory.** Writes `trace.jsonl` in the Section 8 format with every invariant holding: one event per step, flushed immediately, routing events that record who decided, and exactly one terminal event, even after a crash.
 - B6 **Budget capture.** Charges the usage the provider reports, not an estimate.
 - B7 **Budget cutoff.** Checked before each model call. Alerts that are not yet classified get `budget_exhausted`.
 - B8 **Failsafes.** `max_steps` and a no-progress detector, each ending the run with its own terminal status.
 - B9 **report.json.** Validates against `report.schema.json` with one row per alert, and is still written when the run crashes.
 - B10 **All three targets.** Runs end to end with the staff `[scan]` blocks.
-

Part C: evidence and README (3)

- C1 **README.** Following it from a fresh clone (Section 10), we can run your agent end to end on each target with both of your configs, and the low-budget run shows the cutoff.
- C2 **Results.** For each target, the counts of label 1, label 0 and `null`. On OWASP, precision and recall against the benchmark labels, and how you used them to refine the agent (metrics are reported for teaching purposes only; their values are not graded). One sequence diagram of one alert's trajectory, drawn from one of your traces.

C3 Limitations and alternatives. A comparison with at least two other architectures (fixed workflow, plan-and-execute, supervisor/worker) on cost, latency, coordination and failure behaviour. Analyse them; do not build them.

Grading notes:

- Grades assess implementation only; the design report, README and traces provide supporting evidence. Classification correctness and performance (including accuracy, precision and recall) are evaluated only for teaching purposes and do not affect grades.
- A missing mechanism cannot earn its point.
- We check B7 with your `configs/low.toml`.

10 README

Your `README.md` tells us how to run your agent. We follow it from a fresh clone of your repository, point your agent at each target checkout, and run it end to end. How you install and invoke the agent is up to you; the README only has to let us do it without asking you.

It must give:

1. **Setup** from a fresh clone: every command to install your dependencies, Python 3.12 and Bandit 1.9.4, and the environment variables your agent reads (names only, never values).
2. **The run command**, in the form of Section 5, with `--input`, `--output` and `--config`.
3. **Two config files**, committed in your repository, that validate against `config.schema.json`:
 - `configs/full.toml`: the full budget you consider right for a complete run;
 - `configs/low.toml`: a `max_tokens` low enough that the budget runs out partway through the Radicale run, so at least one row of `report.json` has `status budget_exhausted`.

Put the Radicale `[scan]` block in both. For each target we replace the `[scan]` block with that target's required block (Section 4) and keep everything else.

We run your agent six times: on each of the three targets, with each of the two configs. Each run must write `report.json` and `trace.jsonl` (Sections 6 and 8). Anything else in the README is up to you.

11 Deliverables

- Source code, and a lockfile or equivalent that pins dependencies.
- `README.md` as Section 10 describes, with `configs/full.toml` and `configs/low.toml`.
- The design report (Part A plus C2 and C3), as `REPORT.md` or a PDF.
- The unedited `report.json` and `trace.jsonl` of every run your report cites, with the config used. No API keys in the repository.
- A contribution statement that says who did what.

12 Academic integrity and AI tools

AI assistants may be used to learn, debug your own code, review a design, or provide scoped autocomplete. Do not hand the assignment to an agent and submit the result. You remain responsible for every line you submit. Every team member must be able to explain the entire submission; the individual exams test this.

You may not use agents to generate or modify the target code. The guideline on LLM use is strictly enforced: use LLMs to learn, not to do the assignment for you.